CHAPTER 6

Building an Ethereum DApp

In the previous chapter we learned how to programmatically interact with Bitcoin and Ethereum blockchains using JavaScript. We also touched on how to create and deploy Ethereum smart contracts. In this chapter we will take our blockchain application programming to the next level by learning how to develop and deploy a DApp based on the Ethereum blockchain. As part of creating this DApp, we will be setting up a private Ethereum network and then we will use this network as the underlying blockchain for our DApp. This DApp will have its business logic in an Ethereum smart contract, and this logic will be executed using a web application connecting to private Ethereum network. This way, we intend to cover all aspects of Ethereum application development–from setting up nodes and networks, to creating and deploying a smart contract, to executing smart contract functions using client applications.

The DApp

Before we jump into developing the DApp, we need to define the use case for the DApp. We also need to define the various components that will be part of our DApp. So, let's first do this.

© Bikramaditya Singhal, Gautam Dhameja, Priyansu Sekhar Panda 2018
B. Singhal et al., *Beginning Blockchain*, https://doi.org/10.1007/978-1-4842-3444-0_6

The use case for our DApp is a polling application that can allow voters to vote on a poll published in the public domain. Voting using a centralized system is not very reliable, as it exposes a single point of data corruption and failure. So, the goal of our DApp is to enable decentralized polling. This way every voter is in control of their vote and each vote is processed on every node on the blockchain so there is no way to tamper with the vote data. While this can be easily done using the public Ethereum blockchain, to make our exercise interesting we will deploy our polling DApp on a private Ethereum network, and for that we will set up the private network too. Sounds interesting? Let's do this.

The first step will be to set up a private Ethereum network. Then, for hosting the business logic and poll results, we will create a smart contract that will be deployed on this private Ethereum network. To interact with this smart contract, we will create a front-end web application using the web3 library. That's it.

As per the plan just described, our DApp development exercise will have the following steps:

1. Setting up a private Ethereum network
2. Creating a smart contract for polling functionality
3. Deploying the smart contract to the private network
4. Creating a front-end web app to interact with the smart contract

In the following sections, we will be looking, in detail, at each of the steps mentioned.

Note As mentioned, we can also use the public Ethereum network for this DApp development. In addition to that, we can also use several tools like Metamask and Truffle framework to expedite the development of an Ethereum DApp. These tools, along with various others, allow us to manage our code and deployments in a better way. The reader is encouraged to explore these and other tools to try to find the best combination to create a comfortable and productive development environment for their DApp development. This text is primarily focussed on making the reader understand what goes under the hood when creating an Ethereum DApp, hence all tools providing abstractions on top of the DApp development process are kept out of scope.

Setting Up a Private Ethereum Network

To set up a private Ethereum network, we will need one of the many Ethereum clients available. In simple terms, an Ethereum client is an application that implements the Ethereum blockchain protocol. There are many Ethereum clients available on the Internet today; one of the popular ones is go-ethereum, also known as geth. We will be using geth for our private network set-up. For this exercise, we are using a virtual machine running *Ubuntu Linux version 16.04*.

Install go-ethereum (*geth*)

The first step is to install geth on our local machine. To install geth, we will get the geth executable installer from the official source `https://geth.ethereum.org/downloads/`. This download page at the official geth website lists the installer packages for all major platforms (Windows, macOS, Linux).

Download the installer package for your platform and install geth on your local machine. You can also choose to install geth on a remote (cloud-hosted) server/virtual machine if you do not want to install it on your local machine.

Once geth is successfully installed on your local machine, you can check the installation by running the following command in your terminal/command prompt.

```
geth version
```

Depending on your platform OS and the geth version you have installed, this command should give an output similar to the following:

```
Geth
Version: 1.7.3-stable
Git Commit: 4bb3c89d44e372e6a9ab85a8be0c9345265c763a
Architecture: amd64
Protocol Versions: [63 62]
Network Id: 1
Go Version: go1.9
Operating System: linux
GOPATH=
GOROOT=/usr/lib/go-1.9
```

Create *geth* Data Directory

By default, geth will have its working directory but we will create a custom one so that we can track it easily. Simply create a directory and keep the path to this directory handy.

```
mkdir mygeth
```

Create a *geth* Account

The first thing we need is an Ethereum account that can hold Ether. We will need this account to create our smart contracts and transactions later in the DApp development. We can create a new account using the following command.

```
sudo geth account new --datadir <path to the data directory we
created in the previous step>
sudo geth account new --datadir /mygeth
```

Note We are using sudo to avoid any permission issues.

When you run this command, the prompt will ask for a passphrase to lock this account. Enter and confirm the passphrase and then your geth account will be created. Make sure to remember the passphrase you entered; it will be needed to unlock the account later to sign transactions. The address of this account will be shown on the screen. For us, the address of the generated account is **baf735f889d603f0ec6b1030c91d9033e60525c3**. The following screenshot (Figure 6-1) shows this process.

```
author@testserver:~$ sudo geth account new --datadir /mygeth
Your new account is locked with a password. Please give a password. Do not forget this p
assword.
Passphrase:
Repeat passphrase:
Address: {baf735f889d603f0ec6b1030c91d9033e60525c3}
```

Figure 6-1. *Ethereum account setup with geth*

Notice that we have passed the data directory as the parameter for the create account command. This is to make sure that the file containing the account details is created inside our data directory so that it is easy to

access the account from the context of this directory. If we do not pass the data directory parameter to the geth commands, then it will automatically take the default location of the data directory (which can be different depending on the platform).

Create *genesis.json* Configuration File

After installing geth and creating a new account, the next step is to define the genesis configuration for our private network. As we have seen in the previous chapters, blockchains have a genesis block that acts as the starting point of the blockchain, and all transactions and blocks are validated against the genesis block. For our private network, we will have a custom genesis block and hence a custom genesis configuration. This configuration defines some key values for the blockchain like difficulty level, *gas* limit for blocks, etc.

The genesis configuration for Ethereum has the following format as a JSON object. Each of the keys of this object is a configuration value that drives the network.

```
{
    "config": {
        "chainId": 3792,
        "homesteadBlock": 0,
        "eip155Block": 0,
        "eip158Block": 0
    },
    "difficulty": "2000",
    "gasLimit": "2100000",
    "alloc": {
        "baf735f889d603f0ec6b1030c91d9033e60525c3":
        { "balance": "9000000000000000000" }
    }
}
```

The JSON object is primarily constituted by a config section having values specific to chainId and block numbers related to some of the forks that have taken place. The important parameter to note here is the chainId, which represents the identifier of the blockchain and helps prevent replay attacks. For our private chain, we have opted for a random chainId 3792. You can choose any number here different from the numbers used by main net (1) and the test nets (2, 3, and 4).

The next important parameter is the difficulty. This value defines how difficult it will be to mine a new block. This value is much higher in the Ethereum main network, but for private networks we can choose a relatively smaller value.

Then there is *gasLimit*. This is the total *gas* limit for a block and not a transaction. A higher value generally means more transactions in each block.

Finally, we have the alloc section. Using this configuration, we can prefund Ethereum accounts with the value in wei. As we can see, we have funded the same Ethereum account that we created in the last step, with 9 Ether.

Run the First Node of the Private Network

To run the first node of the private blockchain, let's first copy the JSON from the previous step and save it as a file named **`genesis.json`**. For simplicity, we are saving this file in the same directory that we are using as the data directory for geth.

First, we need to initialize geth with the *genesis.json*. This initialization is needed to set the custom genesis configuration for our private network.

CD to the directory where we have saved the *genesis.json* file is

```
cd mygeth
```

The following command will initialize geth with the custom configuration we have defined.

```
sudo geth --datadir "/mygeth" init genesis.json
```

geth will confirm the custom genesis configuration set-up with the output in the following screen-shot (Figure 6-2).

```
author@testserver:~/mygeth$ sudo geth --datadir "/mygeth" init genesis.json
INFO [02-11|17:45:32] Allocated cache and file handles         database=/mygeth/geth/chaindata cache=16 handles=16
INFO [02-11|17:45:32] Writing custom genesis block
INFO [02-11|17:45:32] Successfully wrote genesis state         database=chaindata              hash=294e0a…1f4aa7
INFO [02-11|17:45:32] Allocated cache and file handles         database=/mygeth/geth/lightchaindata cache=16 handles=16
INFO [02-11|17:45:32] Writing custom genesis block
INFO [02-11|17:45:32] Successfully wrote genesis state         database=lightchaindata              hash=294e0a…1f4aa7
```

Figure 6-2. *Initialize geth with configuration in genesis.json*

Next, we need to run geth using the following command and the parameters. We will look into each of these parameters in detail.

```
sudo geth --datadir "/mygeth" --networkid 8956 --ipcdisable
--port 30307 --rpc --rpcapi "eth,web3,personal,net,miner,admin,
debug" --rpcport 8507 --mine --minerthreads=1 --etherbase=0xbaf
735f889d603f0ec6b1030c91d9033e60525c3
```

Let's look at each of the parameters that we gave to the geth command.

datadir: This is to specify the data directory just like we did in the previous steps.

networkid: This is the identifier of the network, which differentiates our private blockchain with other Ethereum networks. This is similar to the chainId we defined in the genesis.json file but provides another layer of differentiation among networks. As we can see, we have used another custom number for this value.

ipcdisable: With this parameter we have disabled the interprocess communication port for geth so that while running multiple geth instances (nodes) on the same local machine we should not encounter any conflicting issues.

port: We have selected a custom value for the port to interact with geth.

rpc, --rpcapi, --rpcport: These three parameters define the configuration for the RPC API exposed by geth. We want to enable it; we want eth,web3,personal,net,miner,admin,debug geth APIs exposed over RPC; and we want to run it on a custom port 8507.

mine – minerthreads – etherbase: With these three parameters we are instructing geth to start this node as a miner node, limit the miner process threads to only one (so that we do not consume a lot of CPU power), and send the mining rewards to the Ethereum account that we created in the first step.

That's all the configuration we need at this time to run our first geth node for the private network.

When we run this command with all the parameters, geth will give the following output (as in the screenshot shown in Figure 6-3).

```
author@testserver:~/mygeth$ sudo geth --datadir "/mygeth" --networkid 8956 --ipcdisable --port 30307 --rpc --rpcapi "eth,web3,personal,net,miner,a
dmin,debug" --rpcport 8507 --mine --minerthreads=1 --etherbase=0xbaf735f889d603f0ec6b1030c91d9033e60525c3
INFO [02-11|18:00:55] Starting peer-to-peer node               instance=Geth/v1.7.3-stable-4bb3c89d/linux-amd64/go1.9
INFO [02-11|18:00:55] Allocated cache and file handles         database=/mygeth/geth/chaindata cache=128 handles=1024
WARN [02-11|18:00:55] Upgrading database to use lookup entries
INFO [02-11|18:00:55] Initialised chain configuration          config="{ChainID: 3792 Homestead: 0 DAO: <nil> DAOSupport: false EIP150: <nil> EIP1
55: 0 EIP158: 0 Byzantium: <nil> Engine: unknown}"
INFO [02-11|18:00:55] Database deduplication successful        deduped=0
INFO [02-11|18:00:55] Disk storage enabled for ethash caches   dir=/mygeth/geth/ethash count=3
INFO [02-11|18:00:55] Disk storage enabled for ethash DAGs     dir=/home/author/.ethash count=2
INFO [02-11|18:00:55] Initialising Ethereum protocol           versions="[63 62]" network=8956
INFO [02-11|18:00:55] Loaded most recent local header          number=0 hash=294e0a…1f4aa7 td=2000
INFO [02-11|18:00:55] Loaded most recent local full block      number=0 hash=294e0a…1f4aa7 td=2000
INFO [02-11|18:00:55] Loaded most recent local fast block      number=0 hash=294e0a…1f4aa7 td=2000
INFO [02-11|18:00:55] Regenerated local transaction journal    transactions=0 accounts=0
INFO [02-11|18:00:55] Starting P2P networking
INFO [02-11|18:00:57] UDP listener up                          self=enode://e03b50e9b1b2579904f2bbdff7dd0826bd4e4eb2e225c1d1cb1a765195474d7418f3e8
fbfeefd55bd85722973d17626f0e53208c62e38d1099bb583e702b3b48@[::]:30307
INFO [02-11|18:00:57] RLPx listener up                         self=enode://e03b50e9b1b2579904f2bbdff7dd0826bd4e4eb2e225c1d1cb1a765195474d7418f3e8
fbfeefd55bd85722973d17626f0e53208c62e38d1099bb583e702b3b48@[::]:30307
INFO [02-11|18:00:57] HTTP endpoint opened: http://127.0.0.1:8507
INFO [02-11|18:00:57] Transaction pool price threshold updated price=18000000000
INFO [02-11|18:00:57] Starting mining operation
INFO [02-11|18:00:57] Commit new mining work                   number=1 txs=0 uncles=0 elapsed=142µs
INFO [02-11|18:00:59] Generating DAG in progress               epoch=0 percentage=0 elapsed=793.271ms
INFO [02-11|18:01:00] Generating DAG in progress               epoch=0 percentage=1 elapsed=1.592s
```

Figure 6-3. *Geth run first node*

Note the UDP listener up log statement in the output.

```
INFO [02-11|18:00:57] UDP listener up
self=enode://e03b50e9b1b2579904f2bbdff7dd0826bd4e4eb2e
225c1d1cb1a765195474d7418f3e8fbfeefd55bd85722973d1762
6f0e53208c62e38d1099bb583e702b3b48@[::]:30307
```

This contains the address of the node we just started. To connect other nodes to this node we will need this address. Let's keep it noted at some place. The following line has the extracted address from the previous log statement.

```
enode://e03b50e9b1b2579904f2bbdff7dd0826bd4e4eb2e225c1d1cb1a7
65195474d7418f3e8fbfeefd55bd85722973d17626f0e53208c62e38d1099
bb583e702b3b48@[::]:30307
```

Note the **[::]** before the port number we defined in the command. Let's replace this with the local host IP address if we are running the other node on the same machine, or else replace it with the external IP address of the machine. As we are going to run the other network node on the same

machine (for development purposes), we will replace it with the localhost IP address. So, the address of the first node will finally be

enode://e03b50e9b1b2579904f2bbdff7dd0826bd4e4eb2e225c1d1cb1a765195474d7418f3e8fbfeefd55bd85722973d17626f0e53208c62e38d1099bb583e702b3b48@127.0.0.1:30307

Run the Second Node of the Network

There is no network with just one node; it should at least have two nodes. So, let's run another geth instance on the same machine, which will interact with the node we just started, and both these nodes together will form our Ethereum private network.

To run another node, first of all we need another directory that can be set as the data directory of the second node. Let us create one.

```
mkdir mygeth2
```

Now, we will initialize this node also with the same genesis.json configuration we created for the first node. Let's create another copy of this genesis.json file and save it in the new directory we created earlier. Let's also CD to this directory. Now, let's initialize the genesis configuration for the second node.

```
sudo geth --datadir "/mygeth2" init genesis.json
```

And, we will get a similar output as we got for the first node. See the screenshot below (Figure 6-4).

```
author@testserver:~/mygeth2$ sudo geth --datadir "/mygeth2" init genesis.json
WARN [02-11|18:19:15] No etherbase set and no accounts found as default
INFO [02-11|18:19:15] Allocated cache and file handles         database=/mygeth2/geth/chaindata cache=16 handles=16
INFO [02-11|18:19:15] Writing custom genesis block
INFO [02-11|18:19:15] Successfully wrote genesis state         database=chaindata               hash=294e0a…1f4aa7
INFO [02-11|18:19:15] Allocated cache and file handles         database=/mygeth2/geth/lightchaindata cache=16 handles=16
INFO [02-11|18:19:15] Writing custom genesis block
INFO [02-11|18:19:15] Successfully wrote genesis state         database=lightchaindata          hash=294e0a…1f4aa7
```

Figure 6-4. *Geth initialize configuration for second node*

Now our second node is also initialized with the genesis configuration. Let's run it.

For running the second node, we will pass a few different parameters to the geth command. This second node will not run as a miner, so we will skip the last three parameters from the command that we gave to the first node. Also, we want to expose the geth console while running this node, so we will add a parameter for that. The command for running the second node will be

```
sudo geth --datadir "/mygeth2" --networkid 8956 --ipcdisable
--port 30308 --rpc --rpcapi "eth,web3,personal,net,miner,admin,
debug" --rpcport 8508 console
```

As we can see, the data directory and ports have been changed for the second node. We also have added the console flag to the command so we can get the geth console for this node.

When we run this command, the second node will also start running and we will see the following output in the terminal (Figure 6-5).

```
author@testserver:~/mygeth2$ sudo geth --datadir "/mygeth2" --networkid 8956 --ipcdisable --port 30308 --rpc --rpcapi "eth,web3,p
ersonal,net,miner,admin,debug" --rpcport 8508 console
WARN [02-11|18:20:27] No etherbase set and no accounts found as default
INFO [02-11|18:20:27] Starting peer-to-peer node               instance=Geth/v1.7.3-stable-4bb3c89d/linux-amd64/go1.9
INFO [02-11|18:20:27] Allocated cache and file handles         database=/mygeth2/geth/chaindata cache=128 handles=1024
WARN [02-11|18:20:27] Upgrading database to use lookup entries
INFO [02-11|18:20:27] Initialised chain configuration          config="{ChainID: 3792 Homestead: 0 DAO: <nil> DAOSupport: false E
IP150: <nil> EIP155: 0 EIP158: 0 Byzantium: <nil> Engine: unknown}"
INFO [02-11|18:20:27] Disk storage enabled for ethash caches   dir=/mygeth2/geth/ethash count=3
INFO [02-11|18:20:27] Disk storage enabled for ethash DAGs     dir=/home/author/.ethash count=2
INFO [02-11|18:20:27] Initialising Ethereum protocol           versions="[63 62]" network=8956
INFO [02-11|18:20:27] Database deduplication successful        deduped=0
INFO [02-11|18:20:27] Loaded most recent local header          number=0 hash=294e0a…1f4aa7 td=2000
INFO [02-11|18:20:27] Loaded most recent local full block      number=0 hash=294e0a…1f4aa7 td=2000
INFO [02-11|18:20:27] Loaded most recent local fast block      number=0 hash=294e0a…1f4aa7 td=2000
INFO [02-11|18:20:27] Regenerated local transaction journal    transactions=0 accounts=0
INFO [02-11|18:20:27] Starting P2P networking
INFO [02-11|18:20:29] UDP listener up                          self=enode://e7ba0ca600eb92e0d665a95edca226dc824230cab7b0bdc403524
32522075d555b38afa9033deea367377f5b5c53ae0ada377c9795f2840b73ffa1c579433e060[::]:30308
INFO [02-11|18:20:29] RLPx listener up                         self=enode://e7ba0ca600eb92e0d665a95edca226dc824230cab7b0bdc403524
32522075d555b38afa9033deea367377f5b5c53ae0ada377c9795f2840b73ffa1c579433e060[::]:30308
INFO [02-11|18:20:29] HTTP endpoint opened: http://127.0.0.1:8508
Welcome to the Geth JavaScript console!

instance: Geth/v1.7.3-stable-4bb3c89d/linux-amd64/go1.9
 modules: admin:1.0 debug:1.0 eth:1.0 miner:1.0 net:1.0 personal:1.0 rpc:1.0 txpool:1.0 web3:1.0

>
```

Figure 6-5. *Geth run second node*

At this time, both our geth nodes are running but they do not know about each other. If we run the **admin.peers** command on the geth console of the second node, we will get an empty array as the result (Figure 6-6).

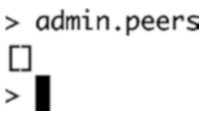


Figure 6-6. *Geth console–check for peers*

This means that the nodes are not connected to each other. Let's connect the nodes. To do this, we will send the **admin.addPeer()** command on the geth console of the second node with the node address of the first node as the parameter. Remember we noted the address of the first node after running it. Let's run this command in the second node's geth console.

```
admin.addPeer("enode://e03b50e9b1b2579904f2bbdff7dd0826bd4e4e
b2e225c1d1cb1a765195474d7418f3e8fbfeefd55bd85722973d17626f0e5
3208c62e38d1099bb583e702b3b48@127.0.0.1:30307")
```

And as soon as we run this command on the second node, it returns true. Also, after a few seconds it starts synchronization with the first node. The following screen shot (Figure 6-7) shows this output from the console of the second node.

```
> admin.addPeer("enode://e03b50e9b1b2579904f2bbdff7dd0826bd4e4eb2e225c1d1cb1a765195474d741
8f3e8fbfeefd55bd85722973d17626f0e53208c62e38d1099bb583e702b3b48@127.0.0.1:30307")
true
> INFO [02-11|18:34:46] Block synchronisation started
INFO [02-11|18:34:46] Imported new state entries              count=1 elapsed=31µs proces
sed=1 pending=0 retry=0 duplicate=0 unexpected=0
INFO [02-11|18:34:47] Imported new block headers              count=192 elapsed=1.329s nu
mber=192 hash=0939a1…01bc65 ignored=0
INFO [02-11|18:34:47] Imported new block receipts             count=192 elapsed=3.419ms b
ytes=768 number=192 hash=0939a1…01bc65 ignored=0
INFO [02-11|18:34:47] Imported new block headers              count=192 elapsed=306.568ms
 number=384 hash=d4236a…0a25ef ignored=0
```

Figure 6-7. *Geth console–add peer node*

Both our nodes are now connected and our private Ethereum network is up. To further verify this, we will run the **`admin.peers`** command again on the second node and this time we will see the JSON array with an object showing the first node as the peer (Figure 6-8).

```
> admin.peers
[{
    caps: ["eth/63"],
    id: "e03b50e9b1b2579904f2bbdff7dd0826bd4e4eb2e225c1d1cb1a765195474d7418f3e8fbfeefd55bd
85722973d17626f0e53208c62e38d1099bb583e702b3b48",
    name: "Geth/v1.7.3-stable-4bb3c89d/linux-amd64/go1.9",
    network: {
      localAddress: "127.0.0.1:53790",
      remoteAddress: "127.0.0.1:30307"
    },
    protocols: {
      eth: {
        difficulty: 120652076,
        head: "0x511dfe69b3e05360ba5deb073e6c34601d6e7266298ccf4c4407aa6683db9285",
        version: 63
      }
    }
}]
```

Figure 6-8. *Geth console–check for peers (again)*

The following screen shot shows the terminal windows of both the nodes we've set up. On the left is the first node, which is also a miner node, and as we can see it is constantly mining new blocks. The second node is on the right and we can see it is synchronizing with the first node. The screenshot (Figure 6-9) is too small to read because of too

much information in it, but it just captures and shows the logs from both Ethereum nodes side by side.

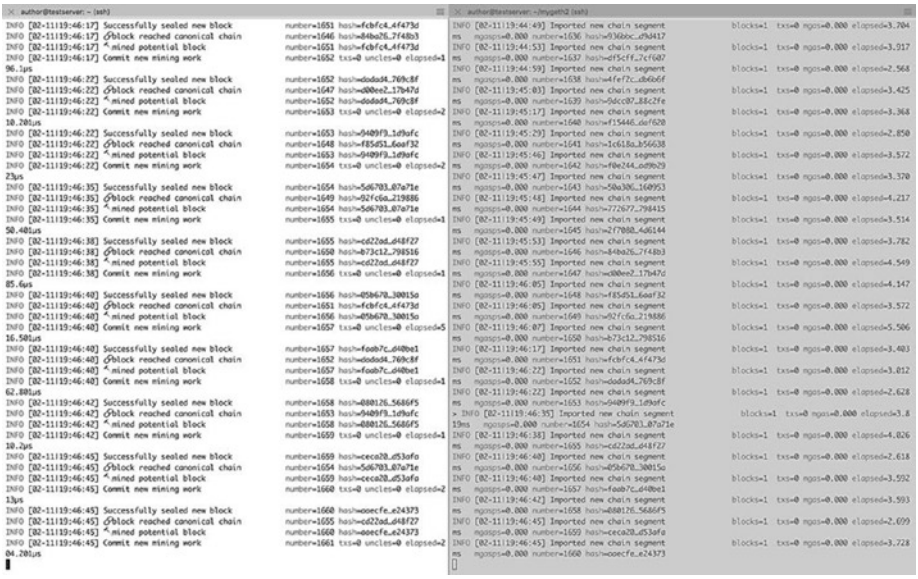


Figure 6-9. *Geth logs from both Ethereum nodes*

Now that both nodes are peers to each other in the network, we have a working private Ethereum blockchain with two nodes. We also have an Ethereum account that is set as a miner and also is prefunded by some Ether amount. We can now create more accounts and pass Ether among them on this private blockchain.

In this section we learned how to set up a private Ethereum network with two nodes. This can be any number of nodes; we just need to follow the same process for each new node. In case of remote nodes, we should be careful about specifying the right IP addresses of the remote machines and we should also make sure that the required ports are opened if there is a firewall preventing traffic to the machines.

Creating the Smart Contract

Now that we have the private Ethereum network set up and working, we can move on to the next step of creating a smart contract for the polling functionality of our DApp. We will then deploy this contract to our private network. We will follow the same steps of creating and deploying a smart contract as we did in the last chapter.

Let's fire up the Remix online IDE and code our smart contract in Solidity.

The following Solidity code snippet shows the smart contract we have coded for the polling functionality.

```
pragma solidity ^0.4.19;

contract Poll {
    event Voted(
        address _voter,
        uint _value
    );

    mapping(address => uint) public votes;

    string pollSubject = "Should coffee be made tax free? Pass
    1 for yes OR 2 for no in the vote function.";

    function getPoll() constant public returns (string) {
        return pollSubject;
    }

    function vote(uint selection) public {
        Voted(msg.sender, selection);

        require (votes[msg.sender] == 0);
        require (selection > 0 && selection < 3);
        votes[msg.sender] = selection;
    }
}
```

Now, let's analyze this contract source code to understand what we have done here. As we can see, the name of the contract is **Poll**.

The next line of code is

```
event Voted(
    address _voter,
    uint _value
);
```

The preceding code snippet is basically declaring a smart contract event that takes two parameters: one is of the type of Ethereum address and another is of the type of unsigned integer. We have created this event so that we can capture who has voted what in the poll. We will come back to this later.

Next, we have

```
mapping(address => uint) public votes;
```

The preceding line of code declares a mapping of Ethereum addresses and unsigned integers. This is the data store where we will be storing the voters' addresses and their chosen value for the vote.

Then we have the following:

```
string pollSubject = "Should coffee be made tax free? Pass 1
for yes OR 2 for no in the vote function.";
function getPoll() constant public returns (string) {
    return pollSubject;
}
```

The preceding code snippet first declares a string for the polling subject. In this we are asking a question of the voters. And then we have a function that can return the value of this string so that voters can query what the poll is about.

And finally, we have the function that implements the voting functionality.

```
function vote(uint selection) public {
    Voted(msg.sender, selection);

    require (votes[msg.sender] == 0);
    require (selection > 0 && selection < 3);
    votes[msg.sender] = selection;
}
```

Examine closely each line of the preceding snippet.

First, as soon as we enter this function, we are raising the voted event we created with the values of the sender's address (voter) and the value he has chosen.

Next, we are limiting one vote per voter by checking if the value of the vote is zero for the corresponding address in the mapping. The *require* statement is used to check conditions based on user inputs.

And then we are also limiting, by using the require statement, the value of the selection to either 1 or 2. 1 is a yes and 2 is a no. And we have passed these instructions in the pollSubject string so that the voters know what to do.

The screenshot in Figure 6-10 shows the smart contract in Remix,

We compiled this contract code using Remix and we took the ABI and byte code for the contract so that we can deploy it to our private network. We copied the bytecode and ABI from the respective sections in the details popup of the Remix compile tab–exactly how we did this in the previous chapter.

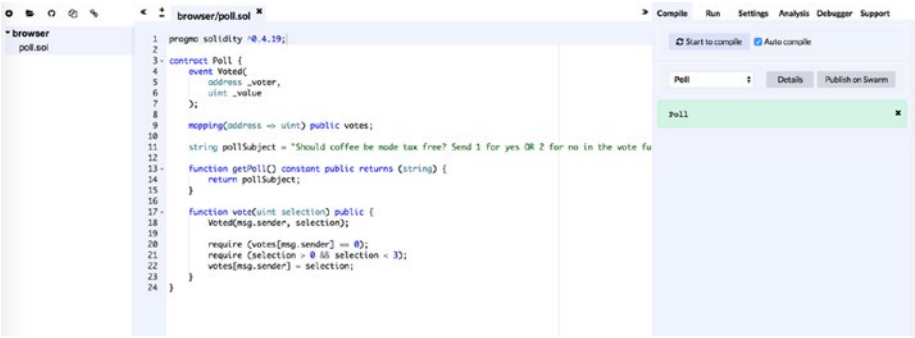


Figure 6-10. *Smart contract editing in Remix online Solidity editor*

The ABI of the contract is

```
[
    {
        "constant": true,
        "inputs": [
            {
                "name": "",
                "type": "address"
            }
        ],
        "name": "votes",
        "outputs": [
            {
                "name": "",
                "type": "uint256"
            }
        ],
        "payable": false,
        "stateMutability": "view",
        "type": "function"
    },
```

```
    {
        "constant": true,
        "inputs": [],
        "name": "getPoll",
        "outputs": [
            {
                "name": "",
                "type": "string"
            }
        ],
        "payable": false,
        "stateMutability": "view",
        "type": "function"
    },
    {
        "anonymous": false,
        "inputs": [
            {
                "indexed": false,
                "name": "_voter",
                "type": "address"
            },
            {
                "indexed": false,
                "name": "_value",
                "type": "uint256"
            }
        ],
        "name": "Voted",
        "type": "event"
    },
```

```
    {
        "constant": false,
        "inputs": [
            {
                "name": "selection",
                "type": "uint256"
            }
        ],
        "name": "vote",
        "outputs": [],
        "payable": false,
        "stateMutability": "nonpayable",
        "type": "function"
    }
]
```

And the byte code for the contract is

```
{
    "linkReferences": {},
    "object": "6060604052608060405190810160405280605081526020017f53686f756c6420636f6666656520626520d616465207461782066726565 3f2081526020017f53656e64203120666f7220796573204f52203220666f72206e6f20696e20746881526020017f6520766f74652066756e6374696f6e2e000000000000000000000000000000008152506001908051906020019061009c9291906100ad565b5034156100a857600080fd5b610152565b828054600181600116156101000203166002900490600052602060002090601f016020900481019282601f10610ee57805160ff1916838001178555561011c565b828001600101855582156101c579182015b828111561011b57825182559160200191906001019061010056 5b5b509050610129919061012d565b5090565b61014f91905b8082111561014b576000816000905550600101610133565b5090565b90565b61037380610161600039600f30060606040526
```

```
00436106100575760003 57c0100000000000000000000000000000000000
0000000000000000000000000900463ffffffff1680630121b93f146
1005c57806303c322781461007f578063d8bff5a51461010d575b600
080fd5b341561006757600080fd5b61007d600480803590602001909
190505061015a565b005b341561008a57600080fd5b6100926102735
65b604051808060200182810382528381815181526020019150805 19
060200190808360005b838110156100d2578082015181840152602
0810190506100b7565b505050509050908101906 01f1680156100ff5
780820380516001836020036101000a031916815260200191505b509
250505060405180910390f35b341561011857600080fd5b610144600
480803573ffffffffffffffffffffffffffffffffffffffff1690602
0019091905050610 31b565b604051808281526020019150506040518
0910390f35b7f4d99b957a2bc29a30ebd96a7be8e68fe50a3c701db2
8a91436490b7d53870ca43382604051808373ffffffffffffffffffff
ffffffffffffffffffff1673ffffffffffffffffffffffffffffffffff
ffffffff16815260200182815260200192505050604051809103 90a
160008060003373ffffffffffffffffffffffffffffffffffffffff1
673ffffffffffffffffffffffffffffffffffffffff1681526020019
081526020016000205414151561021257600080fd5b6000811801 56
102225750600381105b151561022d57600080fd5b806000803373fff
fffffffffffffffffffffffffffffffffff1673ffffffffffffffff
ffffffffffffffffffffffff16815260200190815260200160002 08
190555050565b61027b610333565b60018054600181600116156101 0
0020316600290048060 1f0160208091040260200160405190810160 4
052809291908181526020018280546001816001161561010002031 66
0029004801561031157806 01f106102e657610100808354040283529
160200191610311565b820191906000526020600020905b815481529
060010190602001808311 6102f457829003601f168201915b5050505
050905090565b600602052806000526040600020600091509050548
1565b602060405190810160405280600081525090560 0a165627a7a7
2305820ec7d3e1dae8412ec85045a8eafc248e37ae506802cc008ead
300df1ac81aab490029",
```

"opcodes": "PUSH1 0x60 PUSH1 0x40 MSTORE PUSH1 0x80 PUSH1
0x40 MLOAD SWAP1 DUP2 ADD PUSH1 0x40 MSTORE DUP1 PUSH1 0x50
DUP2 MSTORE PUSH1 0x20 ADD PUSH32 0x53686F756C6420636F666
665652062652065206D6164652074617820667265653F20 DUP2 MSTORE
PUSH1 0x20 ADD PUSH32 0x53656E64203120666F7220796573204F52
203220666F72206E6F20696E207468 DUP2 MSTORE PUSH1 0x20
ADD PUSH32 0x6520766F74652066756E6374696F6E2E00000000
0000000000000000000000000 DUP2 MSTORE POP PUSH1 0x1 SWAP1
DUP1 MLOAD SWAP1 PUSH1 0x20 ADD SWAP1 PUSH2 0x9C SWAP3
SWAP2 SWAP1 PUSH2 0xAD JUMP JUMPDEST POP CALLVALUE ISZERO
PUSH2 0xA8 JUMPI PUSH1 0x0 DUP1 REVERT JUMPDEST PUSH2
0x152 JUMP JUMPDEST DUP3 DUP1 SLOAD PUSH1 0x1 DUP2 PUSH1
0x1 AND ISZERO PUSH2 0x100 MUL SUB AND PUSH1 0x2 SWAP1
DIV SWAP1 PUSH1 0x0 MSTORE PUSH1 0x20 PUSH1 0x0 KECCAK256
SWAP1 PUSH1 0x1F ADD PUSH1 0x20 SWAP1 DIV DUP2 ADD SWAP3
DUP3 PUSH1 0x1F LT PUSH2 0xEE JUMPI DUP1 MLOAD PUSH1 0xFF
NOT AND DUP4 DUP1 ADD OR DUP6 SSTORE PUSH2 0x11C JUMP
JUMPDEST DUP3 DUP1 ADD PUSH1 0x1 ADD DUP6 SSTORE DUP3
ISZERO PUSH2 0x11C JUMPI SWAP2 DUP3 ADD JUMPDEST DUP3 DUP2
GT ISZERO PUSH2 0x11B JUMPI DUP3 MLOAD DUP3 SSTORE SWAP2
PUSH1 0x20 ADD SWAP2 SWAP1 PUSH1 0x1 ADD SWAP1 PUSH2 0x100
JUMP JUMPDEST JUMPDEST POP SWAP1 POP PUSH2 0x129 SWAP2
SWAP1 PUSH2 0x12D JUMP JUMPDEST POP SWAP1 JUMP JUMPDEST
PUSH2 0x14F SWAP2 SWAP1 JUMPDEST DUP1 DUP3 GT ISZERO PUSH2
0x14B JUMPI PUSH1 0x0 DUP2 PUSH1 0x0 SWAP1 SSTORE POP PUSH1
0x1 ADD PUSH2 0x133 JUMP JUMPDEST POP SWAP1 JUMP JUMPDEST
SWAP1 JUMP JUMPDEST PUSH2 0x373 DUP1 PUSH2 0x161 PUSH1
0x0 CODECOPY PUSH1 0x0 RETURN STOP PUSH1 0x60 PUSH1 0x40
MSTORE PUSH1 0x4 CALLDATASIZE LT PUSH2 0x57 JUMPI PUSH1
0x0 CALLDATALOAD PUSH29 0x10000000000000000000000000000
0000000000000000000000000 SWAP1 DIV PUSH4 0xFFFFFFFF

```
AND DUP1 PUSH4 0x121B93F EQ PUSH2 0x5C JUMPI DUP1 PUSH4
0x3C32278 EQ PUSH2 0x7F JUMPI DUP1 PUSH4 0xD8BFF5A5 EQ
PUSH2 0x10D JUMPI JUMPDEST PUSH1 0x0 DUP1 REVERT JUMPDEST
CALLVALUE ISZERO PUSH2 0x67 JUMPI PUSH1 0x0 DUP1 REVERT
JUMPDEST PUSH2 0x7D PUSH1 0x4 DUP1 DUP1 CALLDATALOAD
SWAP1 PUSH1 0x20 ADD SWAP1 SWAP2 SWAP1 POP POP PUSH2 0x15A
JUMP JUMPDEST STOP JUMPDEST CALLVALUE ISZERO PUSH2 0x8A
JUMPI PUSH1 0x0 DUP1 REVERT JUMPDEST PUSH2 0x92 PUSH2
0x273 JUMP JUMPDEST PUSH1 0x40 MLOAD DUP1 DUP1 PUSH1 0x20
ADD DUP3 DUP2 SUB DUP3 MSTORE DUP4 DUP2 DUP2 MLOAD DUP2
MSTORE PUSH1 0x20 ADD SWAP2 POP DUP1 MLOAD SWAP1 PUSH1
0x20 ADD SWAP1 DUP1 DUP4 DUP4 PUSH1 0x0 JUMPDEST DUP4
DUP2 LT ISZERO PUSH2 0xD2 JUMPI DUP1 DUP3 ADD MLOAD DUP2
DUP5 ADD MSTORE PUSH1 0x20 DUP2 ADD SWAP1 POP PUSH2 0xB7
JUMP JUMPDEST POP POP POP POP SWAP1 POP SWAP1 DUP2 ADD
SWAP1 PUSH1 0x1F AND DUP1 ISZERO PUSH2 0xFF JUMPI DUP1
DUP3 SUB DUP1 MLOAD PUSH1 0x1 DUP4 PUSH1 0x20 SUB PUSH2
0x100 EXP SUB NOT AND DUP2 MSTORE PUSH1 0x20 ADD SWAP2 POP
JUMPDEST POP SWAP3 POP POP POP PUSH1 0x40 MLOAD DUP1 SWAP2
SUB SWAP1 RETURN JUMPDEST CALLVALUE ISZERO PUSH2 0x118
JUMPI PUSH1 0x0 DUP1 REVERT JUMPDEST PUSH2 0x144 PUSH1
0x4 DUP1 DUP1 CALLDATALOAD PUSH20 0xFFFFFFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFFFFFF AND SWAP1 PUSH1 0x20 ADD SWAP1 SWAP2
SWAP1 POP POP PUSH2 0x31B JUMP JUMPDEST PUSH1 0x40 MLOAD
DUP1 DUP3 DUP2 MSTORE PUSH1 0x20 ADD SWAP2 POP POP PUSH1
0x40 MLOAD DUP1 SWAP2 SUB SWAP1 RETURN JUMPDEST PUSH32
0x4D99B957A2BC29A30EBD96A7BE8E68FE50A3C701DB28A91436490B7
D53870CA4 CALLER DUP3 PUSH1 0x40 MLOAD DUP1 DUP4 PUSH20 0xF
FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF AND PUSH20 0xFFFF
FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF AND DUP2 MSTORE PUSH1
0x20 ADD DUP3 DUP2 MSTORE PUSH1 0x20 ADD SWAP3 POP POP POP
```

```
PUSH1 0x40 MLOAD DUP1 SWAP2 SUB SWAP1 LOG1 PUSH1 0x0 DUP1
PUSH1 0x0 CALLER PUSH20 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
FFFFFFFFFF AND PUSH20 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
FFFFFF AND DUP2 MSTORE PUSH1 0x20 ADD SWAP1 DUP2 MSTORE
PUSH1 0x20 ADD PUSH1 0x0 KECCAK256 SLOAD EQ ISZERO ISZERO
PUSH2 0x212 JUMPI PUSH1 0x0 DUP1 REVERT JUMPDEST PUSH1 0x0
DUP2 GT DUP1 ISZERO PUSH2 0x222 JUMPI POP PUSH1 0x3 DUP2
LT JUMPDEST ISZERO ISZERO PUSH2 0x22D JUMPI PUSH1 0x0 DUP1
REVERT JUMPDEST DUP1 PUSH1 0x0 DUP1 CALLER PUSH20 0xFFFFFF
FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF AND PUSH20 0xFFFFFFFFFF
FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF AND DUP2 MSTORE PUSH1 0x20
ADD SWAP1 DUP2 MSTORE PUSH1 0x20 ADD PUSH1 0x0 KECCAK256
DUP2 SWAP1 SSTORE POP POP JUMP JUMPDEST PUSH2 0x27B PUSH2
0x333 JUMP JUMPDEST PUSH1 0x1 DUP1 SLOAD PUSH1 0x1 DUP2
PUSH1 0x1 AND ISZERO PUSH2 0x100 MUL SUB AND PUSH1 0x2
SWAP1 DIV DUP1 PUSH1 0x1F ADD PUSH1 0x20 DUP1 SWAP2 DIV
MUL PUSH1 0x20 ADD PUSH1 0x40 MLOAD SWAP1 DUP2 ADD PUSH1
0x40 MSTORE DUP1 SWAP3 SWAP2 SWAP1 DUP2 DUP2 MSTORE PUSH1
0x20 ADD DUP3 DUP1 SLOAD PUSH1 0x1 DUP2 PUSH1 0x1 AND
ISZERO PUSH2 0x100 MUL SUB AND PUSH1 0x2 SWAP1 DIV DUP1
ISZERO PUSH2 0x311 JUMPI DUP1 PUSH1 0x1F LT PUSH2 0x2E6
JUMPI PUSH2 0x100 DUP1 DUP4 SLOAD DIV MUL DUP4 MSTORE SWAP2
PUSH1 0x20 ADD SWAP2 PUSH2 0x311 JUMP JUMPDEST DUP3 ADD
SWAP2 SWAP1 PUSH1 0x0 MSTORE PUSH1 0x20 PUSH1 0x0 KECCAK256
SWAP1 JUMPDEST DUP2 SLOAD DUP2 MSTORE SWAP1 PUSH1 0x1
ADD SWAP1 PUSH1 0x20 ADD DUP1 DUP4 GT PUSH2 0x2F4 JUMPI
DUP3 SWAP1 SUB PUSH1 0x1F AND DUP3 ADD SWAP2 JUMPDEST POP
POP POP POP POP SWAP1 POP SWAP1 JUMP JUMPDEST PUSH1 0x0
PUSH1 0x20 MSTORE DUP1 PUSH1 0x0 MSTORE PUSH1 0x40 PUSH1
0x0 KECCAK256 PUSH1 0x0 SWAP2 POP SWAP1 POP SLOAD DUP2
JUMP JUMPDEST PUSH1 0x20 PUSH1 0x40 MLOAD SWAP1 DUP2 ADD
```

```
    PUSH1 0x40 MSTORE DUP1 PUSH1 0x0 DUP2 MSTORE POP SWAP1
    JUMP STOP LOG1 PUSH6 0x627A7A723058 KECCAK256 0xec PUSH30
    0x3E1DAE8412EC85045A8EAFC248E37AE506802CC008EAD300DF1AC81A
    AB49 STOP 0x29 ",
    "sourceMap": "26:576:0:-;;;167:103;;;;;;;;;;;;;;;;;;;;;;;;;
    ;;;;;;;;;;;;;;;;;;i;;;:-;;26:576;;;;;;;;;;;;;;;;;;;;;;;;;;
    ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
    ;;;;;;;;;;;;;;;;;;;;;;i;;;:-;;;;;;o;;;:-;;;;;;;;;;;;;;;;
    ;;;;;;;;;;;;;;o;;;:-;;;;;;;"
}
```

With these values our smart contract is ready to be deployed.

Important Note There may be some improvements that can be done to make this contract more secure and performance (*gas*) friendly. The Solidity code should not be taken as reference. Detailed discussion on Solidity best practices is out of scope for this text. For Solidity best practices, we recommend following the official solidity documentation and Solidity-specific texts.

Deploying the Smart Contract

In this section we will deploy the smart contract we developed in the last section to the private Ethereum network we have created. The process of deploying the smart contract is the same as what we did in the previous chapter. The only difference is that this time we are deploying the contract to a private network instead of a public one. In this chapter too, we are using the same web3.js library for Ethereum programming using JavaScript. We recommend the reader to go through the previous chapter if they have not already done that.

Setting up *web3* Library and Connection

First of all, we will install the web3 library in a node.js application. This is exactly how we did it in the last chapter. This node.js application will be used to deploy the smart contract.

```
npm install web3@1.0.0-beta.28
```

After installation, let's first initialize and instantiate the *web3* instance.

```
var Web3 = require('web3');
var web3 = new Web3(new Web3.providers.HttpProvider('ht
tp://127.0.0.1:8507'));
```

Note that this time, our HTTP provider for the web3 instance has changed to a local endpoint instead of a public INFURA endpoint, which we used in the last chapter. This is because we are now connecting to our local private network. Also note that the port we are using is **8507**, which is what we provided in the **--rpcport** parameter when we set up the first node of our private network. This means we are connecting to the first node of the network from our web3 instance.

Deploy the Contract to the Private Network

Now that we have our smart contract and its details, we will prepare a web3 contract object with the details of this contract, and then we will deploy this contract to the Ethereum blockchain by calling the deploy method on the contract object.

We need to create an object of the web3.eth.Contract class that can represent our contract. The following code snippet creates a contract instance with the ABI of our contract as an input to the constructor.

```
var pollingContract = new web3.eth.Contract([
    {
        "constant": true,
        "inputs": [
            {
                "name": "",
                "type": "address"
            }
        ],
        "name": "votes",
        "outputs": [
            {
                "name": "",
                "type": "uint256"
            }
        ],
        "payable": false,
        "stateMutability": "view",
        "type": "function"
    },
    {
        "constant": true,
        "inputs": [],
        "name": "getPoll",
        "outputs": [
            {
                "name": "",
                "type": "string"
            }
        ],
```

```
    "payable": false,
    "stateMutability": "view",
    "type": "function"
},
{
    "anonymous": false,
    "inputs": [
        {
            "indexed": false,
            "name": "_voter",
            "type": "address"
        },
        {
            "indexed": false,
            "name": "_value",
            "type": "uint256"
        }
    ],
    "name": "Voted",
    "type": "event"
},
{
    "constant": false,
    "inputs": [
        {
            "name": "selection",
            "type": "uint256"
        }
    ],
    "name": "vote",
    "outputs": [],
```

```
        "payable": false,
        "stateMutability": "nonpayable",
        "type": "function"
    }
]);
```

Now we need to deploy this contract to the Ethereum network using the web3 library's deploy method. The following code snippet shows how to do this. In this snippet we have added the byte code in the data field of the object passed to the deploy method.

```
pollingContract
    .deploy({
        data: '0x6060604052608060405190810160405280605081526020
        0017f53686f756c6420636f6666656520626520 6d6164652074617
        820667265653f2081526020017f53656e64203120666f722079657
        3204f52203220666f72206e6f20696e20746881526020017f65207
        66f74652066756e6374696f6e2e000000000000000000000000000
        0000081525060019080519060200190 61009c9291906100ad565b5
        034156100a857600080fd5b610152565b828054600181600116156
        1010002031660029004906000526020600020090601f01602090048
        1019282601f106100ee57805160ff191683800117855561011c565
        b828001600101855821561011c579182015b828111561011b578
        2518255916020019190600101906101005 65b5b509050610129919
        061012d565b5090565b61014f91905b8082111561014b576000816
        0009055506001016101335 65b5090565b90565b610373806101616
        000396000f300606060405260043610610057576000357c0100000
        0000000000000000000000000000000000000000000000000000900
        463ffffffff1680630121b93f1461005c57806303c322781461007
        f578063d8bff5a51461010d575b600080fd5b34156100675760008
        0fd5b61007d600480803590602001909190505061015a565b005b3
        41561008a57600080fd5b610092610273565b60405180806020018
        2810382528381815181526020019150805190602001908083836 00
```

```
    05b838110156100d25780820151818401526020810190506100b75
    65b5050505090509080190601f1680156100ff578082038051600
    1836020036101000a03191681526020019150506040518091039
    180910390f35b34156101185760008fd5b610144600480803573f
    ffffffffffffffffffffffffffffffffffffffff169060200190919
    0505061031b565b604051808281526020019150506040518091039
    0f35b7f4d99b957a2bc29a30ebd96a7be8e68fe50a3c701db28a91
    436490b7d53870ca43382604051808373ffffffffffffffffffff
    ffffffffffffffff1673ffffffffffffffffffffffffffffffffff
    ffffffff168152602001828152602001925050506040518091039
    0a160008060003373ffffffffffffffffffffffffffffffffffff
    fff1673ffffffffffffffffffffffffffffffffffffffff1681526
    020019081526020016000205414151561021257600080fd5b60008
    111801561022257506003811 05b151561022d57600080fd5b80600
    0803373ffffffffffffffffffffffffffffffffffffffff1673fff
    ffffffffffffffffffffffffffffffffff16815260200190815
    260200160002081905550505 65b61027b610333565b60018054600
    181600116156101000203166002900480601f01602080910402602
    00160405190810160405280929190818152602001828054600181 6
    001161561010002031660029004801561031157 80601f106102e65
    761010080835404028352916020019161031 1565b8201919060005
    26020600020905b81548152906001019060200180831161 02f4578
    29003601f168201915b5050505050905090565b600060205280600
    052604060002060009150905054815 65b602060405190810160405
    280600081525090560 0a165627a7a72305820ec7d3e1dae8412ec8
    5045a8eafc248e37ae506802cc008ead300df1ac81aab490029'
})
.send({
    from: '0xbaf735f889d603f0ec6b1030c91d9033e60525c3',
    gas: 4700000,
    gasPrice: '20000000000000'
},
```

```
    function(error, transactionHash){
        console.log(error);
        console.log(transactionHash);
    })
    .then(function(contract){
        console.log(contract);
    });
```

Note that we have also used the account that we created during the network setup in the "from" field of the send function. As this account was prefunded with nine Ether and it's also added as the etherbase account for the mining rewards, it has enough Ether to deploy a contract.

The full function to deploy the contract will be

```
var deployContract = function () {
    var pollingContract = new web3.eth.Contract([{
            "constant": true,
            "inputs": [{
                "name": "",
                "type": "address"
            }],
            "name": "votes",
            "outputs": [{
                "name": "",
                "type": "uint256"
            }],
            "payable": false,
            "stateMutability": "view",
            "type": "function"
        },
        {
            "constant": true,
            "inputs": [],
```

```
    "name": "getPoll",
    "outputs": [{
        "name": "",
        "type": "string"
    }],
    "payable": false,
    "stateMutability": "view",
    "type": "function"
},
{
    "anonymous": false,
    "inputs": [{
            "indexed": false,
            "name": "_voter",
            "type": "address"
        },
        {
            "indexed": false,
            "name": "_value",
            "type": "uint256"
        }
    ],
    "name": "Voted",
    "type": "event"
},
{
    "constant": false,
    "inputs": [{
        "name": "selection",
        "type": "uint256"
    }],
```

```
            "name": "vote",
            "outputs": [],
            "payable": false,
            "stateMutability": "nonpayable",
            "type": "function"
        }
    ]);
    pollingContract
        .deploy({
            data: '0x6060604052608060405190810160405280605081526020017f53686f756c6420636f6666656520626520d61646520746178206726565653f2081526020017f53656e642031206f6f7220796573204f52203220666f72206e6f20696e20746881526020017f6520766f74652066756e6374696f6e2e0000000000000000000000000000000081525060019080519060200190610090c9291906100ad565b5034156100a857600080fd5b610152565b828054600181600116156101000203166002900490600052602060002090601f016020900481019282601f106100ee57805160ff191683800117855561011c565b8280016001018555821561011c579182015b828111561011b5782518255916020019190600101906101005650b5090506101299190610012d565b5090565b61014f91905b8082111561014b57600081600090555060010161013356505b5090565b90565b6103738061016160003960006f3006060604052600436106100575760003357c01000000000000000000000000000000000000000000000000000000000900463ffffffff1680630121b93f1461005c57806303c322781461007f578063d8bff5a51461010d575b600080fd5b341561006757600080fd5b61007d600480803590602001909190505061015a565b005b341561008a57600080fd5b6100926100273565b60405180806020018281038252838181518152602
```

```
    0019150805190602001908083836000 5b838110156100d2578
    0820151818401526020810190506100b7565b5050505090509
    0810190601f1680156100ff57808203805160018360200361 0
    1000a031916815260200191505b509250505060405180910390
    0f35b34156101185760008 0fd5b6101446004808035 73fffff
    ffffffffffffffffffffffffffffffffffff169060200190919
    0505061031b565b6040518082815260200191505060405180 9
    10390f35b7f4d99b957a2bc29a30ebd96a7be8e68fe50a3c70
    1db28a91436490b7d53870ca43382604051808373ffffffff
    ffffffffffffffffffffffffffffffff1673ffffffffffffff
    ffffffffffffffffffffffff168152602001828152602001 9
    250505060405180910390a160008060003373ffffffffffff
    ffffffffffffffffffffffff1673ffffffffffffffffffff
    ffffffffffffffffffff168152602001908152602001600 2
    0541415156102125760008 0fd5b60008111801561022575 06
    00381105b151561022d57600080fd5b806000803373ffffff
    ffffffffffffffffffffffffffffffff1673ffffffffffff
    ffffffffffffffffffffffff168152602001908152602 00
    160002081905550505 65b61027b610333565b6001805460018
    160011615610100020316600290048060 1f016020809104026
    0200160405190810160405280929190818152602001828054 6
    0018160011615610100020316600290048015 6103115780601
    f106102e65761010080835404028352916020019161031 1565
    b8201919060005260206000209 05b81548152906001019060 2
    0018083116102f457829003601f168201915b5050505050905
    090565b6000602052806000526040600020600091509050548
    1565b602060405190810160405280600081525090 5600a1656
    27a7a72305820ec7d3e1dae8412ec85045a8eafc248e37ae50
    6802cc008ead300df1ac81aab490029'
})
```

```
        .send({
                from: '0xbaf735f889d603f0ec6b1030c91d9033e6
                0525c3',
                gas: 4700000,
                gasPrice: '20000000000000'
            },
            function (error, transactionHash) {
                console.log(error);
                console.log(transactionHash);
            })
        .then(function (contract) {
            console.log(contract);
        });
};
```

After executing this function from our node.js application, we received the following output:

```
Contract {
  currentProvider: [Getter/Setter],
  _requestManager:
   RequestManager {
     provider: null,
     providers:
      { WebsocketProvider: [Function: WebsocketProvider],
        HttpProvider: [Function: HttpProvider],
        IpcProvider: [Function: IpcProvider] },
     subscriptions: {} },
  givenProvider: null,
  providers:
   { WebsocketProvider: [Function: WebsocketProvider],
     HttpProvider: [Function: HttpProvider],
     IpcProvider: [Function: IpcProvider] },
```

```
_provider: null,
setProvider: [Function],
BatchRequest: [Function: bound Batch],
extend:
 { [Function: ex]
   formatters:
    { inputDefaultBlockNumberFormatter: [Function:
    inputDefaultBlockNumberFormatter],
      inputBlockNumberFormatter: [Function:
      inputBlockNumberFormatter],
      inputCallFormatter: [Function: inputCallFormatter],
      inputTransactionFormatter: [Function:
      inputTransactionFormatter],
      inputAddressFormatter: [Function:
      inputAddressFormatter],
      inputPostFormatter: [Function: inputPostFormatter],
      inputLogFormatter: [Function: inputLogFormatter],
      inputSignFormatter: [Function: inputSignFormatter],
      outputBigNumberFormatter: [Function:
      outputBigNumberFormatter],
      outputTransactionFormatter: [Function:
      outputTransactionFormatter],
      outputTransactionReceiptFormatter: [Function:
      outputTransactionReceiptFormatter],
      outputBlockFormatter: [Function: outputBlockFormatter],
      outputLogFormatter: [Function: outputLogFormatter],
      outputPostFormatter: [Function: outputPostFormatter],
      outputSyncingFormatter: [Function:
      outputSyncingFormatter] },
```

```
utils:
 { _fireError: [Function: _fireError],
   _jsonInterfaceMethodToString: [Function:
   _jsonInterfaceMethodToString],
   randomHex: [Function: randomHex],
   _: [Function],
   BN: [Function],
   isBN: [Function: isBN],
   isBigNumber: [Function: isBigNumber],
   isHex: [Function: isHex],
   isHexStrict: [Function: isHexStrict],
   sha3: [Function],
   keccak256: [Function],
   soliditySha3: [Function: soliditySha3],
   isAddress: [Function: isAddress],
   checkAddressChecksum: [Function: checkAddressChecksum],
   toChecksumAddress: [Function: toChecksumAddress],
   toHex: [Function: toHex],
   toBN: [Function: toBN],
   bytesToHex: [Function: bytesToHex],
   hexToBytes: [Function: hexToBytes],
   hexToNumberString: [Function: hexToNumberString],
   hexToNumber: [Function: hexToNumber],
   toDecimal: [Function: hexToNumber],
   numberToHex: [Function: numberToHex],
   fromDecimal: [Function: numberToHex],
   hexToUtf8: [Function: hexToUtf8],
   hexToString: [Function: hexToUtf8],
   toUtf8: [Function: hexToUtf8],
   utf8ToHex: [Function: utf8ToHex],
   stringToHex: [Function: utf8ToHex],
```

```
      fromUtf8: [Function: utf8ToHex],
      hexToAscii: [Function: hexToAscii],
      toAscii: [Function: hexToAscii],
      asciiToHex: [Function: asciiToHex],
      fromAscii: [Function: asciiToHex],
      unitMap: [Object],
      toWei: [Function: toWei],
      fromWei: [Function: fromWei],
      padLeft: [Function: leftPad],
      leftPad: [Function: leftPad],
      padRight: [Function: rightPad],
      rightPad: [Function: rightPad],
      toTwosComplement: [Function: toTwosComplement] },
   Method: [Function: Method] },
clearSubscriptions: [Function],
options:
 { address: [Getter/Setter],
   jsonInterface: [Getter/Setter],
   data: undefined,
   from: undefined,
   gasPrice: undefined,
   gas: undefined },
defaultAccount: [Getter/Setter],
defaultBlock: [Getter/Setter],
methods:
 { votes: [Function: bound _createTxObject],
   '0xd8bff5a5': [Function: bound _createTxObject],
   'votes(address)': [Function: bound _createTxObject],
   getPoll: [Function: bound _createTxObject],
   '0x03c32278': [Function: bound _createTxObject],
   'getPoll()': [Function: bound _createTxObject],
```

```
    vote: [Function: bound _createTxObject],
    '0x0121b93f': [Function: bound _createTxObject],
    'vote(uint256)': [Function: bound _createTxObject] },
  events:
   { Voted: [Function: bound ],
     '0x4d99b957a2bc29a30ebd96a7be8e68fe50a3c701db28a91436
     490b7d53870ca4': [Function: bound ],
     'Voted(address,uint256)': [Function: bound ],
     allEvents: [Function: bound ] },
  _address: '0x59E7161646C3436DFdF5eBE617B4A172974B481e',
  _jsonInterface:
   [ { constant: true,
       inputs: [Array],
       name: 'votes',
       outputs: [Array],
       payable: false,
       stateMutability: 'view',
       type: 'function',
       signature: '0xd8bff5a5' },
     { constant: true,
       inputs: [],
       name: 'getPoll',
       outputs: [Array],
       payable: false,
       stateMutability: 'view',
       type: 'function',
       signature: '0x03c32278' },
     { anonymous: false,
       inputs: [Array],
       name: 'Voted',
       type: 'event',
```

```
    signature: '0x4d99b957a2bc29a30ebd96a7be8e68fe50a3c70
    1db28a91436490b7d53870ca4' },
  { constant: false,
    inputs: [Array],
    name: 'vote',
    outputs: [],
    payable: false,
    stateMutability: 'nonpayable',
    type: 'function',
    signature: '0x0121b93f' } ] }
```

The output shows the various properties of the contract we deployed to our private network. The most important one is the contract address at which the contract is deployed, which is **0x59E7161646C3436DFdF5eBE617B4A172974B481e**.

The contract ABI and address can be used to call a function on the contract. In the next section we will build a simple web app that will call the vote function of this contract, showcasing how the polling can be done from the front end.

Client Application

As we did in the last chapter, we can use the web3 library to call a function on a smart contract. But, in the last chapter we did that using a node.js application and not in a browser application. In this section, we will be using web3 in a browser application to call our deployed smart contract's vote function.

The simplest web application we can create for this DApp is a single web page with a few text and button controls. For the web page, we can use the following code inside an html file and then run it from a local server. Note that running from a local server and not directly opening the file from the browser is important to load the scripts properly, without facing any browser security issues.

```
<html>

<c>
    <meta charset="UTF-8">
    <title>Beginning Blockchain - DApp demo</title>
    <script src="<source of web3 library from any CDN or local
    file>"></script>
</head>

<body>
    <div>
        <p>
            <strong>Beginning Blockchain</strong>
        </p>
        <p>Hi, Welcome to the Polling DApp!</p>
        <p> </p>
        <p>Get latest poll: 
            <button onclick="getPoll()">Get Poll</button>
        </p>
        <p>
            <div id="pollSubject"></div>
        </p>
        <p>Vote: Yes:
            <input type="radio" id="yes"> No:
            <input type="radio" id="no">
        </p>
        <p>Submit: 
            <button onclick="submitVote()">Submit Vote</button>
        </p>
        </p>
    </div>
```

```
<script>
    if (typeof web3 !== 'undefined') {
        web3 = new Web3(web3.currentProvider);
    } else {
        web3 = new Web3(new Web3.providers.HttpProvider
        ('http://127.0.0.1:8507'));
    }

    function getPoll() {
        var pollingContract = new web3.eth.Contract([{
                "constant": true,
                "inputs": [{
                    "name": "",
                    "type": "address"
                }],
                "name": "votes",
                "outputs": [{
                    "name": "",
                    "type": "uint256"
                }],
                "payable": false,
                "stateMutability": "view",
                "type": "function"
            },
            {
                "constant": true,
                "inputs": [],
                "name": "getPoll",
                "outputs": [{
                    "name": "",
                    "type": "string"
                }],
```

```
        "payable": false,
        "stateMutability": "view",
        "type": "function"
    },
    {
        "anonymous": false,
        "inputs": [{
                "indexed": false,
                "name": "_voter",
                "type": "address"
            },
            {
                "indexed": false,
                "name": "_value",
                "type": "uint256"
            }
        ],
        "name": "Voted",
        "type": "event"
    },
    {
        "constant": false,
        "inputs": [{
            "name": "selection",
            "type": "uint256"
        }],
        "name": "vote",
        "outputs": [],
        "payable": false,
        "stateMutability": "nonpayable",
        "type": "function"
    }
```

```
    ], '0x59E7161646C3436DFdF5eBE617B4A172974B481e');

    pollingContract.methods.getPoll().call().
    then(function (value) {
        document.getElementById('pollSubject').
        textContent = value;
    });
};

function submitVote() {
    var value = 0
    var yes = document.getElementById('yes').checked;
    var no = document.getElementById('no').checked;

    if (yes) {
        value = 1
    } else if (no) {
        value = 2
    } else {
        return;
    }

    var pollingContract = new web3.eth.Contract([{
            "constant": true,
            "inputs": [{
                "name": "",
                "type": "address"
            }],
            "name": "votes",
            "outputs": [{
                "name": "",
                "type": "uint256"
            }],
```

```
            "payable": false,
            "stateMutability": "view",
            "type": "function"
        },
        {
            "constant": true,
            "inputs": [],
            "name": "getPoll",
            "outputs": [{
                "name": "",
                "type": "string"
            }],
            "payable": false,
            "stateMutability": "view",
            "type": "function"
        },
        {
            "anonymous": false,
            "inputs": [{
                    "indexed": false,
                    "name": "_voter",
                    "type": "address"
                },
                {
                    "indexed": false,
                    "name": "_value",
                    "type": "uint256"
                }
            ],
            "name": "Voted",
            "type": "event"
        },
```

```
                {
                    "constant": false,
                    "inputs": [{
                        "name": "selection",
                        "type": "uint256"
                    }],
                    "name": "vote",
                    "outputs": [],
                    "payable": false,
                    "stateMutability": "nonpayable",
                    "type": "function"
                }
            ], '0x59E7161646C3436DFdF5eBE617B4A172974B481e');

            pollingContract.methods.vote(value).send({
                from: '0xbaf735f889d603f0ec6b1030c91d9033e
                60525c3'
            }).then(function (result) {
                console.log(result);
            });
        };
    </script>
</body>

</html>
```

Let's now analyze each of the sections of this HTML file.

In the head section of the HTML document, we have loaded the web3 script from either a CDN source or a local source. This is just like we refer to any other third-party JavaScript library in our web pages (JQuery, etc.)

Then in the body section of the HTML, we have the controls for showing the poll subject and radio and submit buttons to capture user input. The overall web page looks like this (Figure 6-11).

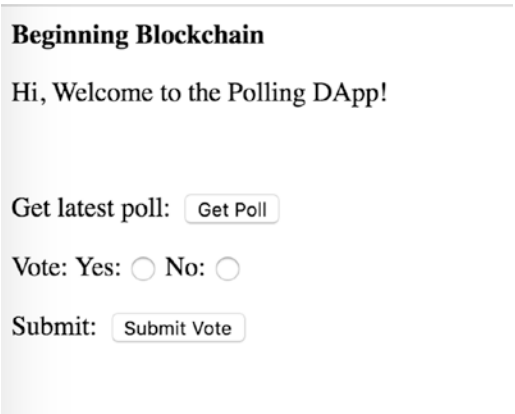


Figure 6-11. *Polling web application view*

What is important is the script section in the body. That's where we are calling the smart contract interaction code. Let's look at it in detail.

```
<script>
        if (typeof web3 !== 'undefined') {
            web3 = new Web3(web3.currentProvider);
        } else {
            web3 = new Web3(new Web3.providers.HttpProvider
            ('http://127.0.0.1:8507'));
        }

        function getPoll() {
            var pollingContract = new web3.eth.Contract([{
                    "constant": true,
                    "inputs": [{
                        "name": "",
                        "type": "address"
                    }],
```

```
    "name": "votes",
    "outputs": [{
        "name": "",
        "type": "uint256"
    }],
    "payable": false,
    "stateMutability": "view",
    "type": "function"
},
{
    "constant": true,
    "inputs": [],
    "name": "getPoll",
    "outputs": [{
        "name": "",
        "type": "string"
    }],
    "payable": false,
    "stateMutability": "view",
    "type": "function"
},
{
    "anonymous": false,
    "inputs": [{
            "indexed": false,
            "name": "_voter",
            "type": "address"
        },
```

```
                    {
                        "indexed": false,
                        "name": "_value",
                        "type": "uint256"
                    }
                ],
                "name": "Voted",
                "type": "event"
            },
            {
                "constant": false,
                "inputs": [{
                    "name": "selection",
                    "type": "uint256"
                }],
                "name": "vote",
                "outputs": [],
                "payable": false,
                "stateMutability": "nonpayable",
                "type": "function"
            }
        ], '0x59E7161646C3436DFdF5eBE617B4A172974B481e');

        pollingContract.methods.getPoll().call().
        then(function (value) {
            document.getElementById('pollSubject').
            textContent = value;
        });
    };
```

```
function submitVote() {
    var value = 0
    var yes = document.getElementById('yes').checked;
    var no = document.getElementById('no').checked;

    if (yes) {
        value = 1
    } else if (no) {
        value = 2
    } else {
        return;
    }

    var pollingContract = new web3.eth.Contract([{
            "constant": true,
            "inputs": [{
                "name": "",
                "type": "address"
            }],
            "name": "votes",
            "outputs": [{
                "name": "",
                "type": "uint256"
            }],
            "payable": false,
            "stateMutability": "view",
            "type": "function"
        },
        {
            "constant": true,
            "inputs": [],
            "name": "getPoll",
```

```
            "outputs": [{
                "name": "",
                "type": "string"
            }],
            "payable": false,
            "stateMutability": "view",
            "type": "function"
        },
        {
            "anonymous": false,
            "inputs": [{
                    "indexed": false,
                    "name": "_voter",
                    "type": "address"
                },
                {
                    "indexed": false,
                    "name": "_value",
                    "type": "uint256"
                }
            ],
            "name": "Voted",
            "type": "event"
        },
        {
            "constant": false,
            "inputs": [{
                "name": "selection",
                "type": "uint256"
            }],
            "name": "vote",
            "outputs": [],
```

```
                "payable": false,
                "stateMutability": "nonpayable",
                "type": "function"
            }
        ], '0x59E7161646C3436DFdF5eBE617B4A172974B481e');

        pollingContract.methods.vote(value).send({
            from: '0xbaf735f889d603f0ec6b1030c91d9033
            e60525c3'
        }).then(function (result) {
            console.log(result);
        });
    };
</script>
```

In the previous script section, first we are initializing the **web3** object with the HTTP provider of the local Ethereum node (if it is not already initialized).

Then, we have two JavaScript functions. One for getting the value of the **pollSubject** string from the smart contract and another for calling the **vote** function of the contract.

The calling of smart contract functions is exactly how we did it in the previous chapter using the **web3.eth.Contract** submodule of the **web3** library.

Note that in the first function **getPoll** we are calling the **call** function on the smart contract instance, while in the second function **submitVote** we are calling **send** on the smart contract instance. That's primarily the difference in the two function calls.

Using the **call** on the **getPoll** function of the smart contract, we are getting the return value of the getPoll function without sending any transaction to the network. We are then showing this value on the UI by assigning it as the text of a UI element.

Next, using **send** on the **vote** function, we are sending a transaction to execute this function on the network and so we have to also define an account that will be used to execute the smart contract function. Following is the output obtained from the submitVote function shown previously, which is basically a transaction receipt.

```
{
        blockHash: '0x04a02dd56c037569eb6abe25e003a65d3366407
        134c90a056f64b62c2d23eb84',
        blockNumber: 4257,
        contractAddress: null,
        cumulativeGasUsed: 43463,
        from: '0xbaf735f889d603f0ec6b1030c91d9033e60525c3',
        gasUsed: 43463,
        logsBloom: '0x00000000000000000000000000000800000000
        00000040000000000000000000200000000000000000000000000
        00000000000000000000000000000000000000000000000000000
        00000000002000000000020000000000000000000000000000000
        00000000000000000000000000000000000000000000000000000
        00000000000000000000000000000000000000000000000000000
        00000000000000000000000000000000000200000000000000000
        00000000000000000000000000000000000000000000000000000
        00000000000000000000000000000000000000000000000000000
        00000000000000000000000000000000000000000000000000000',
        root: '0x58bc4ee0a3025ca3f303df9bb243d052a123026519637
        30c52c88aafe92ebeee',
        to: '0x59e7161646c3436dfdf5ebe617b4a172974b481e',
        transactionHash: '0x434aa9c0037af3367a0d3d92985781c50
        774241ace1d382a8723985efcea73b3',
        transactionIndex: 0,
```

```
        events: {
            Voted: {
                address: '0x59E7161646C3436DFdF5eBE617B4A17
                2974B481e',
                blockNumber: 4257,
                transactionHash: '0x434aa9c0037af3367a0d3d929
                85781c50774241ace1d382a8723985efcea73b3',
                transactionIndex: 0,
                blockHash: '0x04a02dd56c037569eb6abe25e003a65
                d3366407134c90a056f64b62c2d23eb84',
                logIndex: 0,
                removed: false,
                id: 'log_980a1744',
                returnValues: [Result],
                event: 'Voted',
                signature: '0x4d99b957a2bc29a30ebd96a7be8e68f
                e50a3c701db28a91436490b7d53870ca4',
                raw: [Object]
        }
    }
}
```

If we look closely at this output, we see that this also has an events section and it shows the triggering of the **Voted** event that we created in our smart contract.

```
events: {
            Voted: {
                address: '0x59E7161646C3436DFdF5eBE617B4A172
                974B481e',
                blockNumber: 4257,
                transactionHash: '0x434aa9c0037af3367a0d3d929
                85781c50774241ace1d382a8723985efcea73b3',
```

```
            transactionIndex: 0,
            blockHash: '0x04a02dd56c037569eb6abe25e003a6
            5d3366407134c90a056f64b62c2d23eb84',
            logIndex: 0,
            removed: false,
            id: 'log_980a1744',
            returnValues: [Result],
            event: 'Voted',
            signature: '0x4d99b957a2bc29a30ebd96a7be8e68fe
            50a3c701db28a91436490b7d53870ca4',
            raw: [Object]
        }
    }
```

In the preceding code snippet, we've extracted out the events section from the transaction receipt we received in the response of the send transaction to the vote function of our smart contract. As we can see, the events section also shows the return values and the raw values from the function call.

We have now come to the end of our DApp programming exercise. In the previous sections of this chapter, we developed an end-to-end decentralized application on the Ethereum blockchain and we also deployed a private blockchain for our DApp.

The DApp can be used with the public Ethereum network too–a voter has to host a node and they can vote using their existing Ethereum accounts on the public (main) network.

There can be several ways in which the business logic in the smart contract can be enhanced by using different checks and rules.

This programming exercise gives us a basic idea about how to approach development of decentralized applications and the components

that come into the picture during the process. This exercise can be treated as a starting point for Ethereum application development, and the reader is encouraged to explore best practices and more complex scenarios on the subject.

Summary

In this chapter we compiled a programming exercise of developing a decentralized application based on the Ethereum blockchain. We also learned how to set up a private Ethereum network and how to interact with it using the DApp.

References

web3.js Documentation
http://web3js.readthedocs.io/en/1.0/index.html.

Solidity Documentation
https://solidity.readthedocs.org/.

Ethereum Private Networking Tutorial
https://github.com/ethereumproject/go-ethereum/wiki/Private-Networking-Tutorial.